



BIG DATA ANALYTICS CAPSTONE PROJECT

LOAN DEFAULT PREDICTION USING HDFS, HIVE AND SPARK/PYSPARK

HCL GUVI × JAIN UNIVERSITY

PROJECT REPORT

Student Names	Sanjai M, Prashanth P, Sethu Prasath M K Tharun Vikraman K L, Ankith Raghav
Register Numbers	25MSDSR064, 25MSDSR067, 25MSDSR032, 25MSDSR039, 25MSDSR069
Department	M.Sc Data Science and Analytics
Section	B
Semester	III

Abstract

This project develops a Big Data Analytics workflow for Loan Default Prediction using HDFS, Hive and Apache Spark/PySpark. The objective is to analyze loan and customer characteristics associated with default and produce reproducible analytical results. The dataset contains 148,670 records and 34 original attributes, with Status as the target variable: 0 for non-default and 1 for default.

The raw data was stored in HDFS and exposed through a Hive external table. PySpark was then used for data cleaning, missing-value handling, categorical encoding, feature engineering and machine-learning analysis. Numerical missing values were handled using median imputation, while blank categorical values were standardized as Unknown.

Logistic Regression achieved 87.21% accuracy, 87.93% weighted precision, 87.21% weighted recall and 85.86% weighted F1. Random Forest achieved 100% accuracy on the held-out test set. A diagnostic experiment excluding the three highest-importance numerical features reduced Random Forest accuracy to 87.68%, highlighting the strong dependence of the observed result on those variables. These findings are descriptive of this dataset and should not be interpreted as causal conclusions.

1. Introduction

Financial institutions process large volumes of loan applications containing demographic, credit, financial and property-related information. Big Data technologies provide a structured way to store, query and process such data. The assigned capstone project is Loan Default Prediction, with the objective of analyzing loan/customer characteristics associated with default.

The project follows the required flow of DATA SOURCE → HDFS → Hive/Spark → Data Cleaning & Transformation → Distributed Analytics → Results. The supplied capstone guide specifies HDFS, Hive and Spark/PySpark as the core technologies for Loan Default Prediction.

2. Problem Statement

The problem is to build a reproducible Big Data Analytics pipeline that stores loan data in HDFS, organizes it through Hive, performs scalable processing with Spark/PySpark, and identifies patterns associated with loan default.

3. Objectives

- Store and manage the dataset using HDFS.

- Create a Hive database and external table over HDFS.
- Perform meaningful Hive SQL analysis.
- Clean and transform the data using PySpark.
- Engineer numerical and categorical features for machine learning.
- Train Logistic Regression and Random Forest models.
- Analyze default patterns by loan type, region and LTV category.
- Store analytical outputs and predictions back in HDFS.

4. Key Analytics Questions

- What is the observed overall default rate?
- How does default rate vary by loan type?
- How does default rate vary by region?
- How does default rate vary by LTV category?
- Which features receive high Random Forest importance?
- How do the selected classification models perform?

5. Dataset Description

The project uses Loan_Default.csv. It contains 148,670 records and 34 original columns. Status is the target variable, where 0 represents non-default and 1 represents default.

Item	Value
Records	148,670
Original columns	34
Non-default	112,031
Default	36,639
Default rate	24.64%
HDFS raw path	/data/loan_default/raw/Loan_Default.csv
Hive database	loan_default_analytics
Hive table	loan_default

6. Data Dictionary

Column	Meaning
--------	---------

ID	Loan/application identifier
year	Year
loan_limit	Loan limit category
Gender	Gender category
approv_in_adv	Advance approval indicator
loan_type	Loan type
loan_purpose	Loan purpose
Credit_Worthiness	Credit-worthiness category
open_credit	Open-credit indicator
business_or_commercial	Business/commercial indicator
loan_amount	Loan amount
rate_of_interest	Interest rate
Interest_rate_spread	Interest rate spread
Upfront_charges	Upfront charges
term	Loan term
Neg_ammortization	Negative amortization indicator
interest_only	Interest-only indicator
lump_sum_payment	Lump-sum payment indicator
property_value	Property value
construction_type	Construction type
occupancy_type	Occupancy type
Secured_by	Security category
total_units	Number of units
income	Income
credit_type	Credit type
Credit_Score	Credit score
co-applicant_credit_type	Co-applicant credit type
age	Age category
submission_of_application	Submission category
LTV	Loan-to-value ratio
Region	Region
Security_Type	Security type

Status	Target: 0 non-default, 1 default
dtir1	Debt-to-income related ratio

7. Big Data Architecture

Layer	Technology	Purpose
Source	Loan_Default.csv	Original data
Storage	HDFS	Distributed raw/result storage
SQL	Hive	External table and SQL analytics
Processing	Spark/PySpark	Cleaning, transformation and ML
Resource management	YARN	Cluster resource management
Interface	Jupyter	Interactive PySpark development
Results	HDFS	Persist analytical outputs

The reference material requires the project to demonstrate the connection between data source, HDFS, Hive/Spark, transformation, distributed analytics and results.

8. HDFS Data Ingestion

The CSV was copied to the NameNode environment and placed in HDFS at /data/loan_default/raw/Loan_Default.csv. Separate raw, processed and results directories were created under /data/loan_default. HDFS listing commands were used to verify the uploaded dataset.

9. Hive Database and Table

The Hive database loan_default_analytics was created with an external table named loan_default over the raw HDFS directory. The table uses comma-separated fields and skips the CSV header row. The schema matches the actual 34-column dataset.

Hive was used to validate the data and perform grouped analytical queries. This satisfies the capstone requirement for a Hive database/table and meaningful queries.

10. Hive Analytics

Loan Type	Total	Defaults	Default Rate
type1	113,173	25,775	22.77%
type2	20,762	7,172	34.54%
type3	14,735	3,692	25.06%

The loan-type analysis shows different observed default rates across categories. These are descriptive associations within the dataset and do not establish causality.

11. Spark / PySpark Processing

Spark successfully read the CSV directly from HDFS and processed 148,670 records. Blank categorical values were standardized to Unknown. Numerical columns including loan_amount, rate_of_interest, Interest_rate_spread, Upfront_charges, term, property_value, income, Credit_Score, LTV and dtir1 were processed using Spark ML median imputation.

Variable	Missing	Missing %
Upfront_charges	39,642	26.66%
Interest_rate_spread	36,639	24.64%
rate_of_interest	36,439	24.51%
dtir1	24,121	16.22%
property_value	15,098	10.16%
LTV	15,098	10.16%
income	9,150	6.15%

The generated imputed numerical columns contained no null values.

12. Feature Engineering

Categorical variables were transformed using StringIndexer with invalid-value handling and OneHotEncoder. Numerical and encoded categorical variables were combined with VectorAssembler into the features vector. The final feature vector contained 70 features: 10 numerical and 60 encoded categorical dimensions.

Dataset	Records
Training	119,149
Testing	29,521

Total	148,670
-------	---------

13. Machine Learning Models

13.1 Logistic Regression

Logistic Regression was used as a binary classification baseline with Status as the label and the assembled feature vector as input.

13.2 Random Forest

Random Forest was trained with 100 trees, maximum depth 10 and seed 42.

14. Model Evaluation

Metric	Logistic Regression	Random Forest
Accuracy	87.21%	100.00%
Weighted Precision	87.93%	100.00%
Weighted Recall	87.21%	100.00%
Weighted F1	85.86%	100.00%
Default Precision	92.93%	100.00%
Default Recall	52.41%	100.00%
Default F1	67.02%	100.00%

Logistic Regression confusion matrix: TN 21,908; FP 292; FN 3,484; TP 3,837. Random Forest produced 22,200 true negatives and 7,321 true positives on the test set, with zero false positives and false negatives.

The 100% Random Forest result is unusually high and should be interpreted cautiously. Removing the three most influential numerical features—rate_of_interest, Interest_rate_spread and Upfront_charges—reduced Random Forest accuracy to 87.68%. This diagnostic indicates strong dependence of the observed model result on those variables.

15. Random Forest Feature Importance

Feature	Importance
Upfront_charges_imputed	0.2357
Interest_rate_spread_imputed	0.2345
rate_of_interest_imputed	0.1825

credit_type encoded feature	0.1571
property_value_imputed	0.0350
LTV_imputed	0.0315
dtir1_imputed	0.0280

Feature importance represents how the fitted model used variables; it should not be interpreted as causal importance.

16. Business / Analytical Results

16.1 Overall Default Rate

There are 36,639 default records out of 148,670, giving an observed default rate of 24.64%.

16.2 Regional Analysis

Region	Total	Defaults	Default Rate
North-East	1,235	376	30.45%
central	8,697	2,395	27.54%
south	64,016	17,047	26.63%
North	74,722	16,821	22.51%

16.3 LTV Analysis

LTV Category	Total	Defaults	Default Rate
Below 60	32,565	4,534	13.92%
60-69	19,836	2,479	12.50%
70-79	30,059	3,977	13.23%
80-89	26,153	4,822	18.44%
90+	40,057	20,827	51.99%

The 90+ LTV category has an observed default rate of 51.99%, substantially above the other LTV groups. This is an observed association in the project data and should not be treated as causal.

17. Validation and Testing

- Spark input count verified: 148,670.
- Model dataset count verified: 148,670.
- Train/test split verified: 119,149 / 29,521.
- Random Forest prediction output verified: 29,521 rows.

- Loan type, region and LTV outputs were written to and read back from HDFS.
- Status was excluded from the predictor feature columns.
- A feature-removal diagnostic was performed for the unusually high Random Forest result.

18. HDFS Result Outputs

Output	HDFS Location
Random Forest predictions	/data/loan_default/results/random_forest_predictions
Loan type analysis	/data/loan_default/results/loan_type_analysis
Region analysis	/data/loan_default/results/region_analysis
LTV analysis	/data/loan_default/results/ltv_analysis
Model metrics	/data/loan_default/results/model_metrics

19. Challenges and Solutions

Challenge	Solution
Missing numerical values	Median imputation using Spark ML
Blank categorical values	Standardized to Unknown
Categorical variables	StringIndexer + OneHotEncoder
HDFS integration	Spark read directly from hdfs://namenode:8020
Unusually high RF result	Feature-removal diagnostic and cautious interpretation

20. Individual Contribution

The final submission should state the actual team allocation. The technical activities completed in this implementation include Spark/PySpark HDFS ingestion, data cleaning, imputation, categorical encoding, feature engineering, train/test splitting, Logistic Regression, Random Forest, evaluation, feature-importance analysis and writing analytical outputs to HDFS. Each student should list only the work they personally performed and be prepared to explain it. The capstone guide requires individual technical contribution and understanding.

21. GitHub and README

The project repository should contain the PySpark notebook/script, Hive SQL, README, technical screenshots and final documentation. The README should include setup instructions, architecture, HDFS paths, execution steps and results.

GitHub Repository Link: <https://github.com/sanjai-muruganandam/loan-default-prediction-big-data>

22. Technical Evidence

Figure	Screenshot
Figure 1	Docker stack running
Figure 2	HDFS raw dataset
Figure 3	HDFS directory structure
Figure 4	Hive database and table
Figure 5	Hive schema
Figure 6	Hive analytical query
Figure 7	Spark reading HDFS
Figure 8	Spark cleaning
Figure 9	Feature engineering/train-test
Figure 10	Model evaluation
Figure 11	HDFS analytical results
Figure 12	Random Forest predictions in HDFS

These screenshots should be inserted into the final Word document with short captions. The sample project provides a completion template covering HDFS ingestion, Hive, Spark, results, validation, GitHub, individual contributions, challenges and screenshots.

23. Conclusion

The project implemented an end-to-end Big Data Analytics workflow for Loan Default Prediction using HDFS, Hive and Spark/PySpark. The dataset contained 148,670 records and 34 original attributes. HDFS provided distributed storage, Hive provided SQL analysis, and Spark provided preprocessing, feature engineering and machine learning.

The observed default rate was 24.64%. Descriptive analyses showed variation by loan type, region and LTV category. Logistic Regression achieved 87.21% accuracy, while Random Forest achieved 100%

accuracy on the held-out test set. Because the Random Forest result is unusually high, the feature-removal diagnostic was included to provide additional validation context.

The completed pipeline demonstrates the required connection between data storage, SQL analysis, distributed processing, machine learning and HDFS-based result generation.

24. Future Scope

- Fit preprocessing parameters on training data only.
- Use cross-validation and hyperparameter tuning.
- Investigate class imbalance and classification thresholds.
- Compare additional Spark ML classifiers.
- Validate on an independent external dataset.
- Build an interactive dashboard for default-risk exploration.

References

1. Apache Hadoop/HDFS, Apache Hive and Apache Spark concepts used in the classroom Big Data Analytics stack.

2. Loan_Default.csv — project dataset used for this implementation

(Dataset Source : <https://www.kaggle.com/datasets/yasserh/loan-default-dataset?resource=download>)

Appendix A — Technology Stack

Component	Technology
OS/Runtime	Windows + Docker
Storage	HDFS
SQL	Apache Hive
Processing	Apache Spark / PySpark
ML	Spark MLlib
Notebook	Jupyter
Resource management	YARN
Outputs	CSV / Parquet